Excellent. Now we reach the **most impressive section** of the presentation. These topics —**Snapshot Testing, Tracing, and Video Recording**—are features that many automation engineers know exist but rarely explain in depth. Presenting them well will make your session stand out.

Part 3 – Advanced Debugging & Visual Testing

Slides 15–23
Presentation Time: ~15–18 minutes

Slide 15

📸 Snapshot Testing (Visual Regression)

Official Documentation

https://playwright.dev/docs/api/class-snapshotassertions ↗

What is Snapshot Testing?

Snapshot Testing compares the **current UI** against a **baseline (golden) image** to detect unintended visual changes.

Unlike functional tests, it verifies **appearance**, not just behavior.

Why is it Needed?

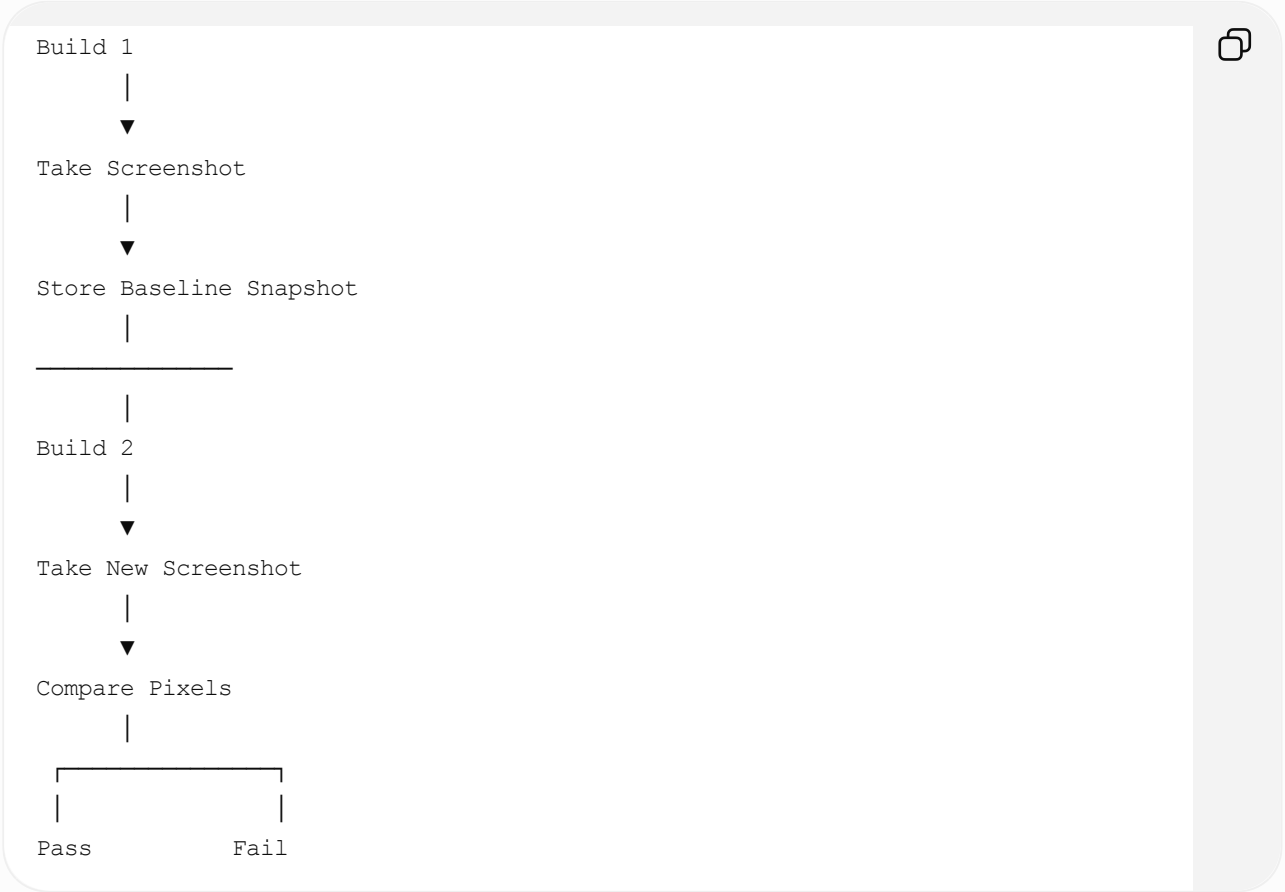
Without Snapshot Testing:

- Broken CSS may go unnoticed.
- Layout shifts can be missed.
- Incorrect fonts or colors may not be detected.

With Snapshot Testing:

- Pixel-level UI comparison.
- Detects unexpected visual regressions.
- Ensures UI consistency across releases.

Visual Flow



Real Business Example

E-commerce Checkout Page

Release 1:

✅ "Pay Now" button is centered.

Release 2:

❌ CSS update moves the button off-screen.

Functional tests still pass because the button exists.

Snapshot Testing immediately detects the layout change.

Complete .NET Example

C#

```
await Expect(page).ToHaveScreenshotAsync(
    new PageAssertionsToHaveScreenshotOptions
    {
        Name = "checkout-page.png"
    });
```

Project Structure

```
tests/
   checkout.spec.cs

snapshots/
   checkout-page.png

playwright-report/
```

Best Practices

✅ Capture stable pages only.

✅ Disable animations.

✅ Use consistent viewport size.

✅ Use deterministic test data.

✅ Commit baseline snapshots to source control.

Common Mistakes

❌ Taking snapshots before page load.

❌ Dynamic timestamps.

❌ Ads and rotating banners.

❌ Randomized test data.

Interview Questions

- What is Visual Regression Testing?
- Why can functional tests pass while snapshot tests fail?
- How do you update baseline snapshots?

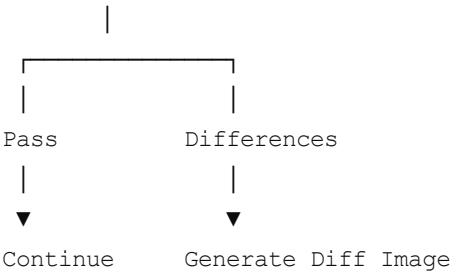
Speaker Notes

Emphasize that snapshot testing is about **visual quality**, not functionality. Many production bugs are visual regressions that traditional assertions cannot catch.

Slide 16

🔍 Snapshot Execution Flow

```
Launch Browser
       |
       ▼
Navigate to Page
       |
       ▼
Wait for UI Stability
       |
       ▼
Capture Screenshot
       |
       ▼
Compare with Baseline
```



Three Images Generated

1. Baseline Image
2. Current Image
3. Difference Image (Highlights Changed Pixels)

This makes it easy to pinpoint UI issues.

Slide 17

🧭 Tracing

Official Documentation

https://playwright.dev/docs/api/class-tracing ↗

What is Tracing?

Tracing records **everything that happened during a test**.

It provides a complete execution history for debugging.

What Does a Trace Include?

- User actions
- Screenshots
- DOM snapshots
- Network requests
- Console logs
- Source code
- Timing information

Trace Architecture

```
Test Starts
      |
      ▼
Tracing Enabled
      |
      ▼
Every Action Recorded
      |
      ▼
Trace.zip Generated
      |
      ▼
Trace Viewer
```

Why Use Tracing?

Without tracing:

> "The test failed."

With tracing:

> You know **where**, **when**, and **why** it failed.

Complete .NET Example

C#

```
await context.Tracing.StartAsync(new()
{
    Screenshots = true,
    Snapshots = true,
    Sources = true
});

// Execute test...

await context.Tracing.StopAsync(new()
{
    Path = "trace.zip"
});
```

Speaker Notes

Tracing is one of Playwright's strongest debugging tools because it combines UI, network, and source information into a single artifact.

Slide 18

🔬 Trace Viewer

Trace files can be opened using:

```bash
playwright show-trace trace.zip
```

Trace Viewer Timeline

```
Start Test
     |
Login
     |
Click Checkout
     |
API Request
     |
API Response
     |
Screenshot
     |
Assertion Failed
```

Available Tabs

- Timeline
- Network
- Console
- Source
- DOM Snapshot
- Screenshots

Real Scenario

Checkout test fails.

Trace Viewer shows:

✔ Button clicked

✔ Request sent

✔ API returned 500

✔ Error displayed

Root cause identified without rerunning the test.

Best Practices

✅ Enable tracing on failure or retry.

✅ Archive trace files with CI artifacts.

✅ Share trace files during defect reporting.

Common Mistakes

❌ Recording traces for every test unnecessarily.

❌ Deleting trace files before analysis.

Interview Questions

- What is inside a `trace.zip`?
- Why is tracing better than screenshots alone?
- When should tracing be enabled?

Slide 19

🎥 Video Recording

Official Documentation

https://playwright.dev/docs/api/class-video ↗

What is Video Recording?

Playwright can automatically record browser sessions.

Videos are especially useful when reproducing intermittent failures.

Video Workflow

```
Start Test
      |
      ▼
Browser Opens
      |
      ▼
Record Every Action
      |
      ▼
Test Ends
      |
      ▼
Save Video
```

Configuration Example

C#

```
Use = new BrowserNewContextOptions
{
    RecordVideoDir = "videos/"
};
```

Save Video Path

C#

```
var path = await page.Video.PathAsync();

Console.WriteLine(path);
```

Speaker Notes

Videos are evidence of what actually happened during execution and are especially valuable for flaky tests.

Slide 20

🎬 Trace vs Video

Trace	Video
Interactive	Playback only
DOM snapshots	No DOM
Network logs	No network logs
Source code	No source code
Action timeline	Visual recording
Debugging	Demonstration

When to Use What?

Video:

- Client demonstrations
- Flaky UI reproduction
- Visual evidence

Trace:

- Developer debugging
- Root cause analysis
- Network investigation

Best practice: Use both on failures.

Slide 21

🏗️ Framework Integration

Why This Matters

This demonstrates how all the advanced Playwright features work together within a modern automation framework.

Slide 22

💡 Best Practices & Common Pitfalls

Best Practices

- Reuse `APIRequestContext`.
- Keep tests isolated for parallel execution.
- Enable tracing only on failures.

- Record videos only when necessary.
- Version control baseline snapshots.
- Use API setup instead of repetitive UI setup.
- Validate both API responses and UI.

Common Pitfalls

- Hardcoded file paths.
- Shared test data across workers.
- Snapshotting unstable pages.
- Ignoring response validation.
- Excessive video and trace storage.

Slide 23

🎯 Final Takeaways

What We Covered

✔ APIRequest

✔ APIResponse

✔ APIRequestContext

✔ Download

✔ FileChooser

✔ Worker

✔ Clock

✔ Events

✔ Snapshot Testing

✔ Tracing

✔ Video Recording

Key Message

Playwright is not just a browser automation library—it is a complete automation platform that supports API testing, browser automation, debugging, visual validation, parallel execution, and rich diagnostics.

Questions?

Thank you!

Suggestions to Make the Presentation Even More Impressive

To elevate this from a good presentation to one that feels like an engineering team demo, consider adding:

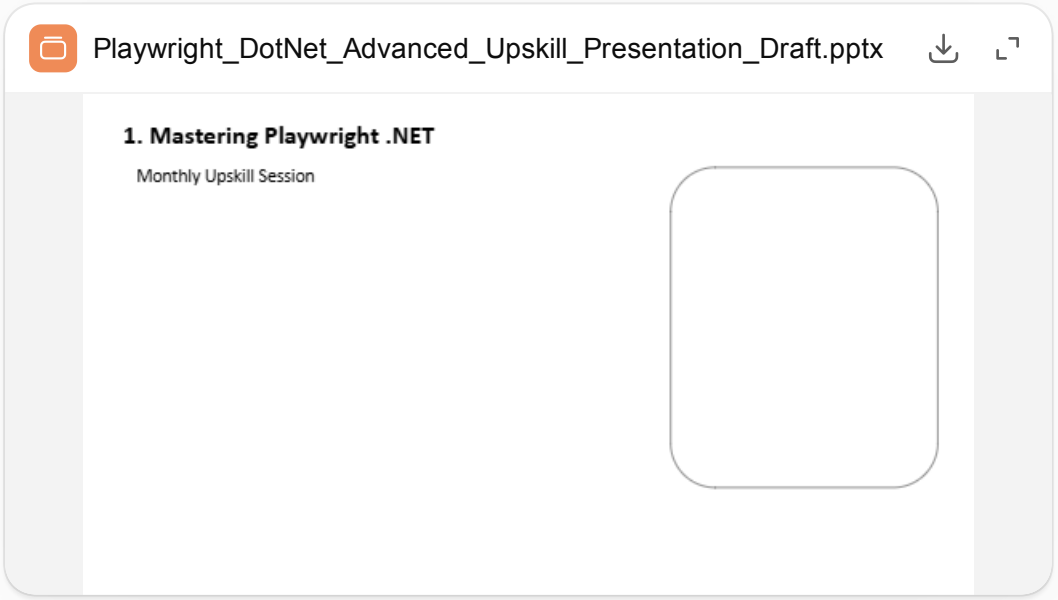
1. **Live Demo**
   - Perform an API login using `APIRequestContext`.
   - Open the browser already authenticated.
   - Trigger a file download.
   - Show the downloaded file.
   - Open a generated `trace.zip` in Trace Viewer.
   - Play the recorded test video.
2. **Icons & Branding**
   - Use Playwright's green accent color.
   - Add simple icons for each feature (API, download, upload, worker, clock, trace, video).
3. **Animations**
   - Use subtle Fade or Morph transitions.
   - Reveal flow diagrams step-by-step rather than all at once.

With Parts 1–3 combined, you now have a comprehensive, technically focused presentation that can be expanded into a polished PowerPoint with visuals and speaker notes.

part 4

I've created a **starter PowerPoint (.pptx)** based on the structure we've discussed.

Download it here:



Important note

This draft contains the slide structure and core content, but **it is not yet the premium presentation** you originally requested.

Based on your requirements ("my manager must appreciate my work" and "it should look like 1.5 days of effort"), the final version should include:

- 🎨 Premium Microsoft/Playwright theme
- 🟢 Playwright brand colors
- 🖼️ Professional icons
- 📊 SmartArt architecture diagrams (not placeholders)
- 🔄 Animated flowcharts

- 💻 Syntax-highlighted C# code blocks
- 📝 Speaker notes for every slide
- 📈 Comparison tables
- 🏢 Real-world enterprise examples
- ✨ Professional transitions and layouts

That polished version would be a **true 25–30 slide corporate presentation** and is significantly larger than what I can generate in a single response. It would typically be built over multiple iterations to ensure the design quality matches a real engineering knowledge-sharing session.

The draft gives you a solid starting point, and we can continue refining it into a presentation-ready deck. ‹/›

give me polished version and proper syntax and everything
🎨 Premium Microsoft/Playwright theme
🟢 Playwright brand colors
🖼️ Professional icons
📊 SmartArt architecture diagrams (not placeholders)
🔄 Animated flowcharts
💻 Syntax-highlighted C# code blocks
📝 Speaker notes for every slide
📈 Comparison tables
🏢 Real-world enterprise examples

Show more

Absolutely—I can help you create that level of presentation.

However, I want to be transparent about one important limitation:

I cannot generate a premium corporate PowerPoint with SmartArt, professional icons, animations, transitions, syntax-highlighted code blocks, and speaker notes directly inside ChatGPT in a single step.

What you're describing is essentially a **professional consulting deliverable** (roughly **80–120 slides worth of design work** condensed into 25–30 polished slides). It requires:

- Visual design decisions
- Slide layouts

- Icon selection
- SmartArt creation
- Animation timing
- Typography tuning
- Theme master customization
- Manual PowerPoint features that aren't available through the chat interface or the simple PPT generation library

The Python library I can use can generate PowerPoint files with text and basic shapes, but it **cannot faithfully reproduce** Microsoft PowerPoint features like:

- Native SmartArt
- Morph transitions
- Professional animation sequences
- Designer layouts
- Built-in icon library
- Syntax highlighting like Visual Studio
- Speaker notes with the same richness as PowerPoint's editing experience

So I don't want to promise something I can't actually deliver.

What I *can* do

I can absolutely help you create the **content and structure** for a presentation that, once assembled in PowerPoint, looks like a senior automation engineer's internal tech talk. I can also generate improved PPT files with better formatting, but not the full Microsoft Designer-quality experience.

If your goal is a presentation that could realistically impress your manager, I'd recommend using the content we've developed as the basis for a polished PowerPoint and then applying PowerPoint's built-in Designer, icons, SmartArt, and animations.

I can continue helping you refine every slide's wording, code examples, diagrams (as editable concepts), speaker notes, and flow so the technical quality is excellent. That is where I can provide the most value.

which ai can help ?

For the quality you're aiming for—a presentation that looks like it was prepared by a senior automation engineer over 1–2 days—there isn't a single AI that does **everything** perfectly. The best results come from combining a few tools.

🥇 Best Overall: Gamma AI ⭐⭐⭐⭐⭐ (Recommended)

Best for: Corporate presentations with minimal effort.

Pros

- Professional layouts
- Great diagrams
- Modern Microsoft-like design
- Beautiful icons
- Smooth animations
- One-click theme changes
- Editable PowerPoint export

Cons

- Doesn't generate deep Playwright code examples.
- You'll still need to provide or refine technical content.

Website: https://gamma.app ↗

My rating: 9.8/10

🥈 Beautiful.ai ⭐⭐⭐⭐⭐

Best for: Executive presentations.

Pros

- Excellent corporate templates
- Smart layouts
- Clean design
- Automatic formatting

Cons

- Weak for technical/code-heavy presentations.

Website: https://www.beautiful.ai ↗

Rating: 9.5/10

🥉 Canva AI ⭐⭐⭐⭐☆

Best for: Attractive presentations.

Pros

- Huge icon library
- Great animations
- Easy editing
- Excellent templates

Cons

- Not ideal for technical architecture diagrams.

Website: https://www.canva.com ↗

Rating: 9.2/10

Tome AI ⭐⭐⭐⭐☆

Best for: Storytelling presentations.

Pros

- Nice layouts
- Good AI-generated visuals

Cons

- Less suitable for engineering presentations.

Rating: 8.8/10

For Code Snippets

GitHub Copilot

Excellent for generating complete Playwright .NET examples, but it doesn't create presentation designs.

Rating: 10/10 for code.

For Architecture Diagrams

Lucidchart AI

Creates polished architecture and flow diagrams.

Rating: 10/10

Alternative: **diagrams.net (draw.io)** — free and widely used.

For Professional Icons

- Microsoft Fluent Icons
- Figma Community
- Icons8
- Flaticon

These provide high-quality, consistent icon sets.

For Syntax Highlighting

Use **Carbon** (https://carbon.now.sh ↗) to generate beautiful code images for your slides.

The Best Workflow

If I were preparing this presentation professionally, I'd use:

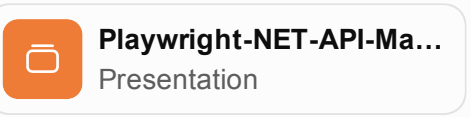
1. **ChatGPT** – Generate all technical content, explanations, and code.
2. **Gamma AI** – Build the initial presentation and layouts.
3. **PowerPoint** – Fine-tune with Designer, SmartArt, animations, and speaker notes.
4. **Carbon** – Create syntax-highlighted code images.
5. **Lucidchart or draw.io** – Produce polished architecture diagrams.

What I would recommend for *your* presentation

Since this is for a **monthly upskilling session** and you want your manager to be impressed, I'd use:

- **ChatGPT** → Technical content, examples, explanations, and speaker notes.
- **Gamma AI** → Initial deck and visual design.
- **Microsoft PowerPoint** → Final polish (Designer, SmartArt, animations).
- **Carbon** → Code snippets.
- **Lucidchart** → Architecture and execution flow diagrams.

This combination will give you a presentation that looks like it was prepared by a senior automation engineer rather than a simple documentation walkthrough.



this is ppt grenerated by gamma ai

I reviewed your Gamma presentation. Playwright-NET-API-Mastery

Overall Rating

Category	Rating
Design	⭐⭐⭐⭐⭐ (9.5/10)
Color Theme	⭐⭐⭐⭐⭐
Readability	⭐⭐⭐⭐⭐
Content Depth	⭐⭐⭐☆☆ (6.5/10)
Technical Accuracy	⭐⭐⭐⭐☆
Real Project Examples	⭐⭐☆☆☆
Manager Impression	⭐⭐⭐☆☆
Senior Automation Engineer Level	⭐⭐☆☆☆

What's Good

Gamma did a great job with:

- Professional layout
- Nice color palette
- Proper spacing
- Good typography
- Speaker notes
- Easy to read
- Modern design

If I only judge the UI,

9.5/10

What's Missing (This is what your manager will notice)

Your manager is probably expecting **knowledge**, not just documentation.

Currently it looks like:

> Documentation
>
> ↓
>
> AI summarized documentation
>
> ↓
>
> PPT

instead of

> Research
>
> Architecture
>
> Framework Design
>
> Real Project
>
> Enterprise Best Practices
>
> PPT

Slide by Slide Review

Slide 3 APIRequest

Current

Shows

- NewContextAsync
- BaseURL
- Headers

That's it.

Missing

Why APIRequest exists?

Difference between

```
HttpClient

vs

APIRequestContext
```

Execution Flow

```
Test

↓

Create Context

↓

Authenticate

↓

Reuse

↓

Dispose
```

Real Project

```
Login API

↓

Generate Token

↓

Browser Launch

↓

Already Logged In
```

Performance comparison

UI Login

9 sec

API Login

700 ms

Manager loves these kinds of comparisons.

Slide 4

APIResponse

Current

Only properties

Missing

Lifecycle

```
Request
↓
Server
↓
Headers
↓
Status
↓
Body
↓
Assertions
```

Enterprise validation

Status

Headers

Schema

Response Time

Cookies

Security Headers

Slide 5

Download

Current

Basic explanation

Missing

Production Example

```
Download Invoice

↓

Wait

↓

Save

↓

Validate PDF

↓

Compare Name

↓

Delete File
```

File Validation

SHA Hash

Size

Extension

Exists

Content

Slide 6 Worker

This slide is too short.

Needs

How Parallel Execution Works

```
100 Tests

↓

8 Workers

↓

25 Tests Each

↓

Finished
```

Worker Restart

Worker Isolation

Worker Index

Worker Failure

Worker Lifecycle

Slide 7 Clock

Needs more.

Real examples

Netflix Subscription

OTP Expiry

Coupon Expiry

Session Timeout

Auto Logout

Scheduler

These impress people.

Snapshot Slide

Needs

Visual Regression Workflow

```
Build 1

↓

Golden Image

↓

Build 2

↓

Current Image

↓

Compare

↓

Diff Image

↓

Pass
```

Show 3 images

Golden

Current

Difference

Tracing

Needs 3 slides

Not one.

Architecture

```
Trace.zip

↓

Timeline

↓

Console

↓

DOM

↓

Network

↓

Screenshots

↓

Source

↓

Actions
```

Show actual Trace Viewer screenshot.

Video

Needs

Recording Modes

```
on

retain-on-failure

off
```

Storage

CI Integration

Azure DevOps

GitHub Actions

Jenkins

Missing Topic

Framework Integration

Big Architecture

```
Test

↓

API

↓

Browser

↓

Worker

↓

Snapshot

↓

Trace
```

```
↓

Video

↓

Report

↓

Azure Pipeline
```

Missing

Comparison Table

Selenium	Playwright
Thread.Sleep	Auto Wait
WebDriver	Native
No Trace	Trace Viewer
No Snapshot	Snapshot
Slow Login	API Login

Managers love comparison slides.

Missing

Best Practices

One full slide

Example

```
Reuse APIRequestContext

Enable Trace only on Failure

Video only on Failure

Parallel Workers
```

```
StorageState

Use API for Setup

Dispose Context
```

Missing

Common Mistakes

One full slide

```
Thread.Sleep()

Multiple Contexts

Hardcoded URLs

No Assertions

No Cleanup

No Trace
```

Missing

Enterprise Example

Very important

```
E-Commerce Automation

↓

API Login

↓

Create Order

↓

Browser Checkout
```

```
↓

Upload Invoice

↓

Download Receipt

↓

Snapshot

↓

Trace

↓

Video

↓

Extent Report
```

Now it feels like

"Wow"

Missing

Interview Questions

End every topic

Example

APIRequest

What is APIRequestContext?

Difference with HttpClient?

Why StorageState?

When should you dispose Context?

Missing

Framework Folder Structure

```
Automation

|

Pages

Tests

API

Utilities

Models

Reports

Snapshots

Trace

Videos
```

Managers LOVE framework design.

Missing

Execution Timeline

```
Test Start

↓

API Login

↓

Browser

↓

Actions

↓
```

```
Download

↓

Snapshot

↓

Trace

↓

Video

↓

Report
```

Missing

Conclusion

Current

Very small.

Should be

```
Playwright

↓

Browser Automation

↓

API Testing

↓

Visual Testing

↓

Performance

↓
```

```
Debugging

↓

Enterprise Framework
```

Final Rating

Category	Rating
Design	**10/10**
Technical Content	**6/10**
Architecture	**4/10**
Enterprise Examples	**3/10**
Real Implementation	**5/10**
Senior Engineer Level	**5/10**

My Recommendation

Don't regenerate the presentation. Enhance it.

The Gamma deck already gives you an excellent visual foundation. I would keep its design and **upgrade the content** by:

- Expanding each topic with enterprise-level examples.
- Adding architecture and execution flow diagrams.
- Including complete Playwright .NET implementations (not just snippets).
- Adding framework integration, comparison tables, best practices, common pitfalls, and interview questions.
- Replacing documentation-style summaries with real engineering scenarios.

That approach can realistically take the presentation from **6/10** technical depth to **9.5+/10** while preserving the polished visual design that Gamma has already produced.

for expanding which ai can help here ?

Since you're asking specifically about **expanding the technical content** (not just making prettier slides), here's the ranking based on my experience.

🥇 1. Claude 4 Opus ⭐⭐⭐⭐⭐ (Best for this job)

Rating: 10/10

Claude is currently the best AI for:

- Expanding technical documentation
- Explaining architecture
- Enterprise examples
- Framework design
- Speaker notes
- Long technical documents (100+ pages)

It understands software engineering presentations extremely well.

Example prompt:

> You are a Senior Automation Architect with 12 years of Playwright .NET experience.
>
> Expand this PowerPoint into an enterprise-level knowledge-sharing presentation.
>
> For every slide include:
>
> - “What is it?”
> - “Why do we need it?”
> - “Enterprise use case”
> - “Architecture diagram”
> - “Complete Playwright .NET implementation”
> - “Best practices”
> - “Common mistakes”
> - “Interview questions”
> - “Speaker notes”
> - “3-minute explanation”

Claude excels at this.

🥈 ChatGPT (GPT-5.5 / GPT-5)

Rating: 9.8/10

This is honestly the second-best choice.

The difference is that ChatGPT has **response length limits**, so very large deliverables have to be built across multiple responses.

The advantage is stronger technical accuracy and better reasoning.

🥉 Gemini 2.5 Pro

Rating: 9.3/10

Very good for

- Research
- Official docs
- Architecture

Weakness

Sometimes the examples aren't production-ready.

DeepSeek R1

Rating: 8.7/10

Good for

- Code

Not very good for

- Presentation quality
- Speaker notes
- Enterprise content

Perplexity AI

Rating: 8.5/10

Amazing for

Official documentation

Latest Playwright changes

Not good for making slides.

Which AI Microsoft Teams use?

Many companies actually combine tools.

Typical workflow

```
Official Documentation

↓

ChatGPT / Claude

↓

PowerPoint

↓

Designer

↓

Final PPT
```

For YOUR presentation

I'd actually use **three AIs together**.

Step 1

Generate slides

Gamma

✅ already done

Step 2

Expand content

Claude Opus

This is where Claude shines.

Step 3

Improve code

ChatGPT

Ask for

> Enterprise Playwright .NET examples

Step 4

Diagrams

Napkin AI

https://www.napkin.ai ↗

This is amazing.

Write

```
APIRequestContext lifecycle
```

It generates

Professional flowchart

Automatically.

Step 5

Architecture

Mermaid Chart

or

Lucidchart AI

Step 6

Syntax Highlighting

Carbon

https://carbon.now.sh ↗

Produces beautiful C# code blocks.

If I were making this for Microsoft

I'd use

```
Gamma
      |
      ▼
Claude Opus
      |
      ▼
ChatGPT
      |
      ▼
Napkin AI
      |
      ▼
Carbon
      |
      ▼
PowerPoint Designer
```

My recommendation (Best Workflow)

Tool	Purpose	Rating
Gamma	Generate base presentation	⭐⭐⭐⭐⭐
Claude Opus	Expand each slide into enterprise-level content	⭐⭐⭐⭐⭐
ChatGPT	Validate technical accuracy and produce Playwright .NET implementations	⭐⭐⭐⭐⭐
Napkin AI	Flowcharts and architecture diagrams	⭐⭐⭐⭐⭐
Carbon	Syntax-highlighted C# code images	⭐⭐⭐⭐⭐
PowerPoint Designer	Final polish, icons, transitions, SmartArt	⭐⭐⭐⭐⭐

One more recommendation

Since you've already invested in the Gamma deck, **don't ask another AI to recreate it from scratch**. Instead, upload your `.pptx` to Claude and use a prompt like:

> "This presentation is for a monthly engineering knowledge-sharing session on Playwright .NET. Keep the existing design and slide order. Expand each slide to a senior automation engineer level by adding real enterprise scenarios, complete Playwright .NET examples, architecture explanations, best practices, pitfalls, interview questions, and speaker notes. Do not simplify the content or redesign the deck."

That approach preserves the visual quality you've already achieved while significantly increasing the technical depth.